XentiQ 1R Technical Documentation

System Overview

This repository contains the source code for the Arduino controller for the XentiQ 1R machine, which communicates with the PIC microcontroller via UART. The Arduino controller is responsible for the following:

- Front-end display and user interface (using the EVE display)
- Stepper motor control and limit switch monitoring
- Profile management
- PIC microcontroller communication
- PLC communication

System Architecture

The codebase is organized into the following modules:

Module	Description
`controllers/`	State machine controllers for different UI screens and workflows
`views/`	Display rendering components for UI elements
`hardware/`	Low-level hardware interfaces (motors, limit switches)
`serial/`	UART communication protocols for PIC interface
`logic/`	Business logic for profile handling and dispensing operations
`gpu/`	Graphics processing utilities for the EVE display

Configuration

System configuration is centralized in **`Config.h`**, which contains all hardware and operational parameters:

Key Configuration Parameters

- **Motor Settings**: Speed, acceleration, and steps-per-unit for X/Y/Z axes
- **Limit Switch Pins**: Hardware pin assignments for safety limits
- **Dispenser Timeouts**: ACK timeout, cycle timeout
- **Profile Defaults**: Origin position, pitch spacing, Z-dip depth, cycle count
- **Security**: Password settings for system access
- **Debug Flags**: Enable/disable logging, screen-only mode, memory monitoring

System States

This document provides visual references for the different states of the system. Most states represent a single `Controller`, which handle the system's logic and state. For simplicity, the "Settings" state diagram is separated from the main state diagram.

Main States

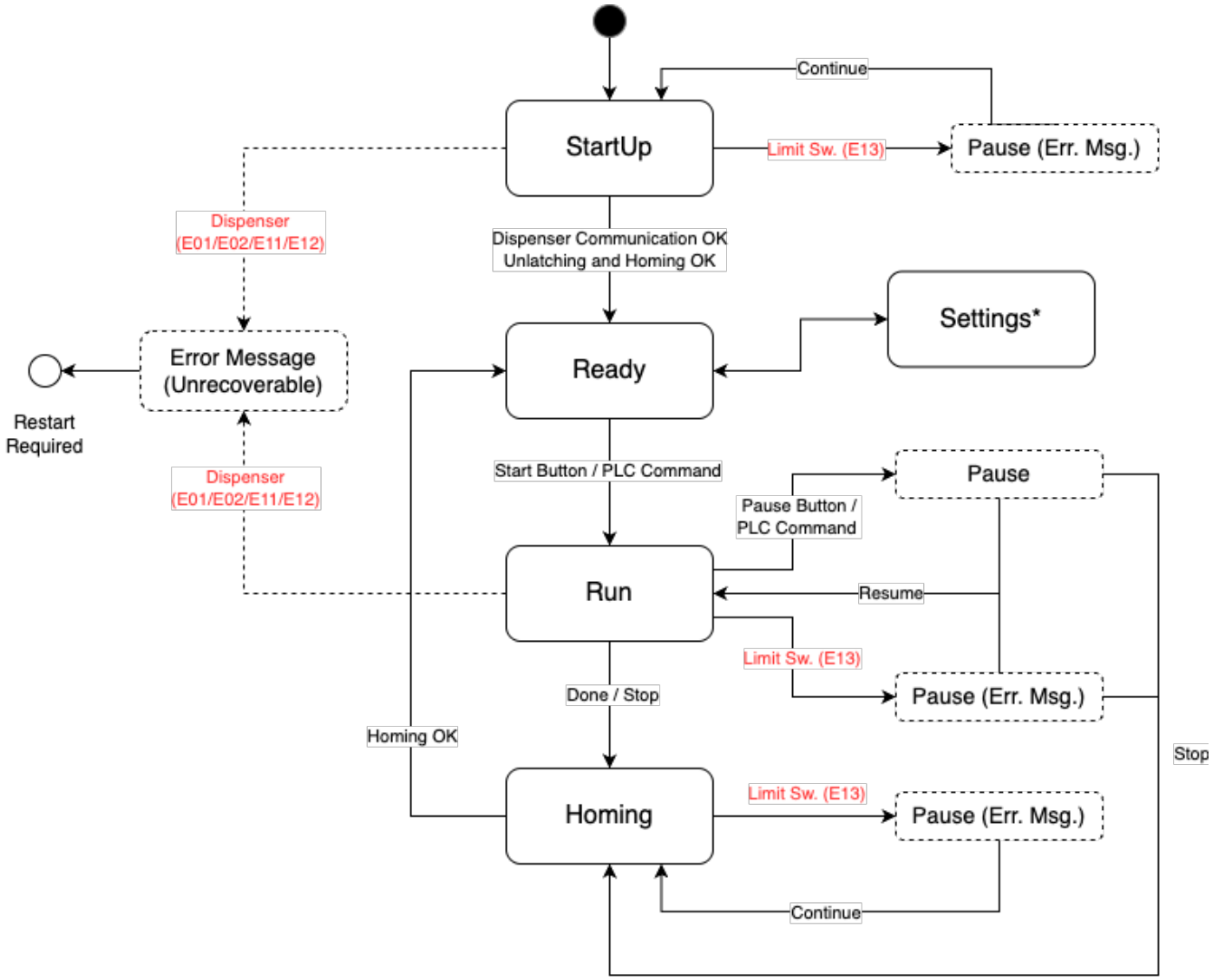

Figure 1: States Diagram

Settings States

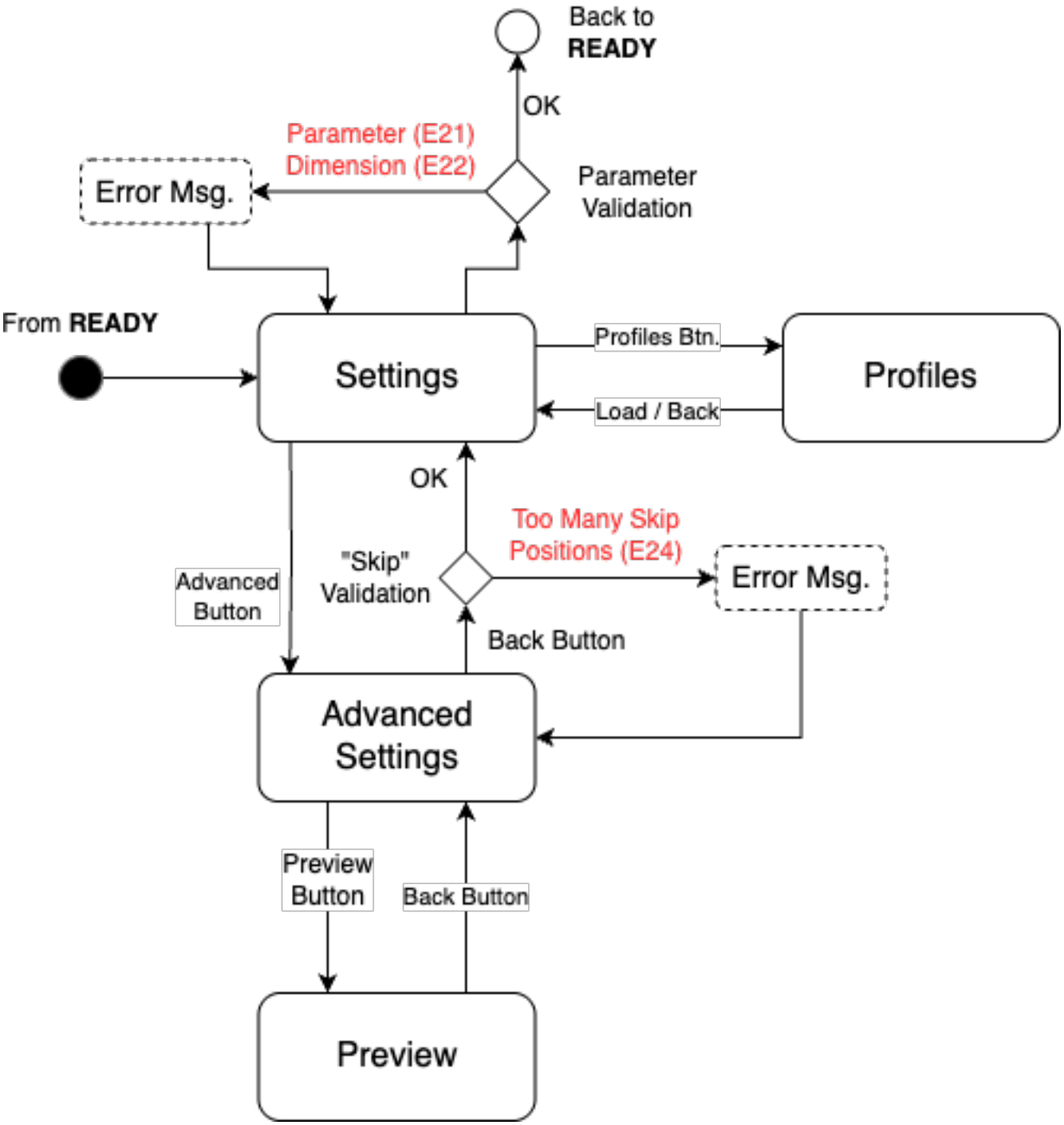

Figure 2: Settings States Diagram

Error Codes

This document lists all error codes and their meanings.

Error Reference Table

Error Code	Explanation
E01 - IR Sensor Error	Passed-through from the PIC.
E02 - IR Marker Error	Passed-through from the PIC.
E11 - Dispenser Error	The PIC did not receive an ACK from the dispenser head within the timeout period.
E12 - Dispenser Timeout Error	The PIC did not receive a “Cycle Complete” signal from the dispenser head within the timeout period.
E13 - Limit Switch Triggered	A limit switch was triggered unexpectedly, indicating a possible movement boundary collision.
E21 - Parameter Error	One or more parameters in the tray configuration are outside the valid range.
E22 - Dimension Error	The tray dimensions calculated from the parameters are outside the valid range.
E23 - Skip Values Error	Skip positions are invalid for current tray configuration.
E24 - Too Many Skip Positions	The total number of skip positions exceeds the maximum allowed.
E99 - Unknown Error	An unknown error occurred. This means the error code is not defined in the code.

Controller Architecture

Controller Interface

At any point in time, one controller is active on the system, handling the logic of the entire system.

The controller might simply be waiting for user input, or running the stepper motors until a certain position is reached, or waiting for the dispenser to finish a dispense cycle.

Controllers share a common interface, which allows them to interact with the rest of the system in a predictable way.

Controllers can signal that another controller should take over by using the `startNewController` callback.

Controller Methods

Method	Caller	Description
`onStart`	ControllerManager	Triggers initial logic when controller becomes active
`onInteraction`	InteractionHandler	Handles button presses and PLC commands
`onStep`	Main Loop	Runs stepper motors, processes dispenser commands/responses, handles internal state machine

Controller Dependencies

Controllers receive the following dependencies during initialization:

- `dispenserHead`: Axis movement and dispenser commands
- `profileManager`: Get/update profiles
- `phost`: Update UI display

Controller Lifecycle

1. **Initialization**: Controller is created with injected dependencies (`dispenserHead`, `profile–Manager`, `phost`)
2. **Activation**: `ControllerManager` calls `onStart()` to trigger initial logic
3. **Event Loop**:
    - `InteractionHandler` calls `onInteraction()` for button presses and PLC commands
    - Main loop continuously calls `onStep()` for motor control, dispenser communication, and state management
4. **Transition**: Controller calls `startNextController(<con_id>)` callback to transfer control to another controller

Dispense Cycle Flow

This document describes the dispense cycle managed by the RunController, which orchestrates the entire dispensing operation through a series of stages from initialization to completion. This process includes:

- Setting the vibration level and duration on the dispenser head
- Priming the dispenser head
- Calculating the most efficient path to fill all required positions
- Controlling the Z axis to “dip” the dispenser head into the tray

The dispense cycle is broken down into **stages**, to ensure that the system can pause and resume the cycle at any point, and to allow for proper error handling.

Stages Overview

The dispense cycle consists of 11 stages that execute sequentially:

1. **Idle** - Initial idle state, all motors stopped
2. **Set Vibration Level** - Configure vibration level on dispenser
3. **Set Vibration Duration** - Configure vibration duration on dispenser
4. **Zero** - Move all axes to zero position (home)
5. **Start Prime** - Send prime command to dispenser
6. **Wait Prime** - Wait for prime cycle to complete
7. **Move** - Move XY axes to target position
8. **Lower Head** - Lower Z axis to dip depth
9. **Start Dispense** - Send dispense command to dispenser
10. **Wait Dispense** - Wait for dispense cycle to complete
11. **Raise Head** - Raise Z axis back to zero position

Error Handling

During any stage, the following errors will trigger a dialog and cause certain actions:

Error Code	Description
E01 - IR Sensor Failure	Pause cycle indefinitely and request user to restart the device
E02 - Marker Not Detected	Pause cycle indefinitely and request user to restart the device
E11 - ACK Error	Pause cycle indefinitely and request user to restart the device
E12 - Cycle Timeout	Pause cycle indefinitely and request user to restart the device
E13 - Limit Switch Triggered	Pause cycle and allow user to resume or stop the dispense cycle

When paused, the user can:

- **Resume**: Continue from the current stage
- **Stop**: Abort the cycle and return to home position